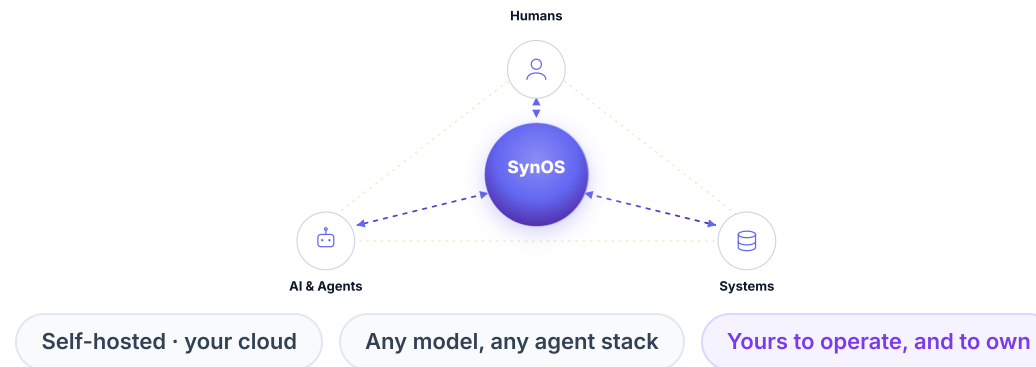


SYNOS

The Human-Agent Operating Layer

The platform an enterprise builds its own AI on, designed for the critical knowledge work that runs the business.

A per-company **AI training and evaluation environment**: company memory, governed access into real systems, somewhere safe to deploy what gets built, and a trace and correction loop over everything that runs. Installed **inside your own infrastructure**, under the AI tools your teams already use. Your systems are profiled where they sit and queried live, so nothing is migrated. Engineering sets the rails once; we land the first workflow, then **hand the controls to the domain experts in ops, finance and service** and step back. Everything it learns belongs to them. **Three engagements live, all three verbally committed to paid contracts and in contracting now.**



PRE-SEED · 2026

THE PREMISE

Every company is going to have to become an AI-native company.

Almost none of them wants to build the infrastructure that takes, and almost none can hire the AI bench it assumes. That is why this layer gets bought rather than built.

THE PRESSURE

NOT OPTIONAL ANY MORE

The mandate is already on the board agenda.

A competitor ships AI features, or leadership asks for the efficiency story. Every company we meet has a programme running. None started it because they wanted one.

the question is no longer whether, it's how

THE GAP

WHAT IT ACTUALLY TAKES

Becoming AI-native is an infrastructure problem.

Context over your systems, governed access, somewhere safe to deploy, a record of what worked. **Every enterprise ends up building the same five things internally, separately and slowly.**

months of platform work before the first useful agent

THE CHOICE

BUILD IT OR BUY IT

Nobody's moat is their AI plumbing.

A manufacturer's edge is manufacturing. A lender's is underwriting. Building the substrate themselves spends their scarcest engineering on **no differentiation at all.**

vendor-built platforms succeed 2x as often as internal builds • MIT

AND IT IS GOING BADLY

95% of enterprise GenAI pilots deliver no P&L impact (MIT). **~40%** of agentic projects cancelled by 2027 (Gartner). The failure is never the model.

WHICH IS THE OPPORTUNITY

The destination isn't in doubt. What none of them has is **somewhere to do it**, that they own.

Models learned the entire internet. **They never learned your company.**

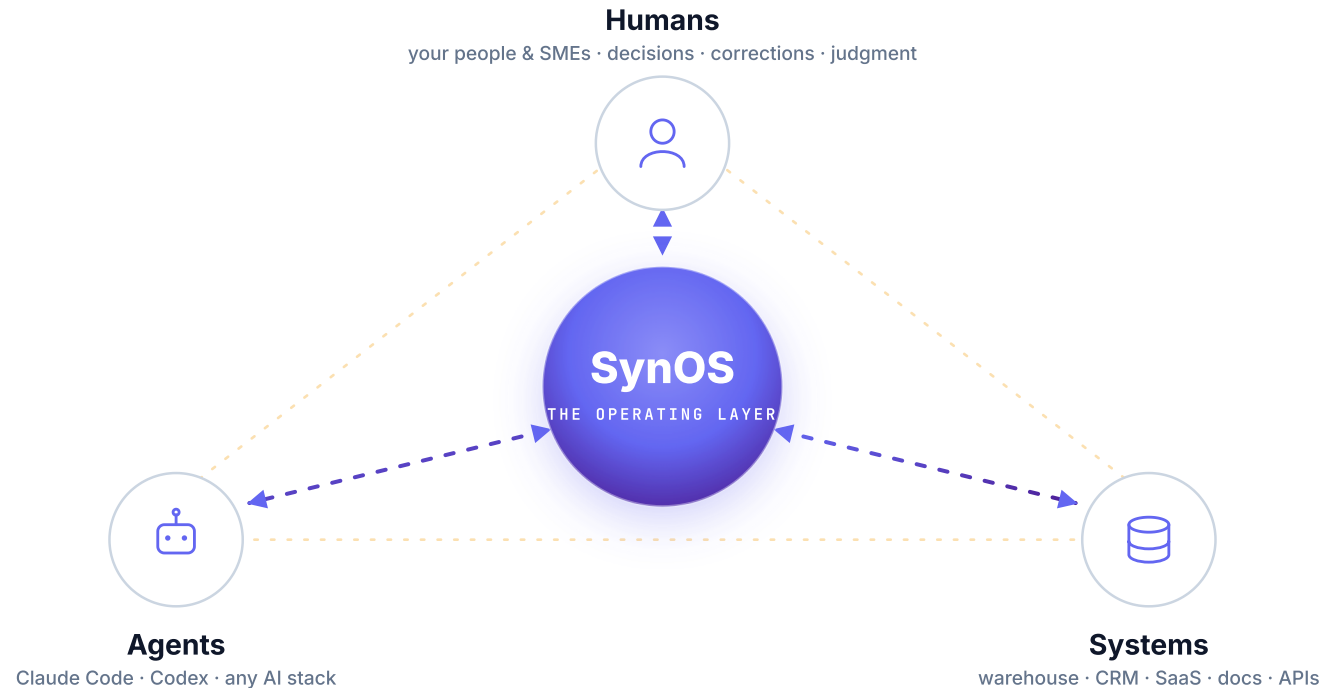
Frontier models are trained on the world's **common** knowledge. But enterprise value lives in what is **not** on the internet. How your company actually operates: its data, its decisions, its tribal knowledge. That's exactly where every agent pilot stalls.

The agents have hands now. They have no company to stand on.

WHERE THEY ARE TODAY

Their people, their AI and their systems are blocked from each other.

Knowledge sits with people. Capability sits in the models. Data sits in the systems. Nothing safely connects the three, so every attempt at AI stops in the same gap.



Everyone selling them AI is selling them a migration first.

The platforms already in the building say the same thing more politely: bring the data to us first. Rip out the legacy stack, move the data, rebuild the pipelines, and **then** you get AI. Two years and a programme budget before a single agent does real work. Many of our buyers are already inside one.

CONSULTANCIES & SYSTEMS INTEGRATORS

The migration is the product

Billable hours are the business model. A multi-year transformation programme is the deliverable; working agents are a downstream promise.

HYPERSCALERS & PLATFORM VENDORS

Your data has to move first

Consumption is the business model. The AI is real, but it only works once the company's data lives on their side of the line.

AI-NATIVE SAAS

A quieter lift-and-shift

No migration programme, but the same trade: your operational data, your corrections and your evals accumulate inside someone else's tenancy.

SynOS comes to the data instead.

Nothing is replaced, nothing is moved, and agents are doing real work in weeks — because the layer installs inside the customer's own infrastructure and reads their systems where they already are. This is not a positioning choice; it is what an in-tenant, model-agnostic architecture actually delivers. **The messy legacy estate stops being the blocker and becomes the asset.**

Evidence: a second enterprise committed on the same template **in weeks, fully air-gapped inside their own infrastructure** — no data left the building, no system was migrated.

One environment under the chaos, built once for the whole enterprise.

Engineering sets the rails once. After that everyone, non-coders included, builds and runs real work on top of them, from the AI tools they already use.

YOUR TEAMS KEEP THEIR OWN AI TOOLS, ALL CONNECTED THROUGH MCP, THE OPEN STANDARD AGENTS USE TO CALL TOOLS

Claude Code
CLI · engineers

Codex / GPT
CLI · engineers

Cursor
IDE · power users

In-house agents
custom stacks

SynOS Apps
sandboxed

Always-on agents
scheduled

MCP · one interface · any model

SYNOS — THE HUMAN-AGENT OPERATING LAYER

Self-hosted · governed · model- & tool-agnostic

 THE LOOP'S MEMORY · SELF-IMPROVING

Company Brain
A living map of how your company operates: entities, relations, citations. Sharper with every run and every correction.



The Learning Loop
Every human correction is reviewed, promoted, and reaches every agent.



Compounding Skills
Authored in plain English; shared, versioned, forked across teams.



Agent-Native Storage
A governed database agents safely write to and read from.



Safe Build & Deploy
Apps and agents ship through sandboxes: scanned, proxied, live URLs.

GUARDRAILS & OBSERVABILITY

role-based access

audit on every call

build scan

egress proxy

kill-switch

every run traced & collected

CONNECTED TO YOUR EXISTING SYSTEMS, NOT REPLACING THEM

Warehouse · BigQuery

CRM · Salesforce

Docs · Notion · Drive

Slack · Email

GitHub · Tickets

Internal APIs

Making an enterprise AI-native is a cluster of hard problems. We take the ones nobody else wants.

Governed integration into systems never built for it. Safe environments for AI to act on real data. And underneath both, the one we are built to own: **turning messy, un-moved, on-prem data into a usable per-company AI environment.**

MAKE BAD DATA USABLE IN PLACE

Profiling & entity resolution, agent-driven.

Legacy systems with no clean schema, thirty years of drift, no migration allowed. Agents profile the systems where they sit and resolve the same entity across all of them.

no warehouse project · no schema rewrite · nothing leaves the building

KEEP IT ALIVE

A company brain that survives the data shifting underneath it.

Static extraction rots in weeks. The brain re-profiles, re-resolves and re-cites as the systems change, which is why it gets sharper with use instead of going stale.

living context · cited · continuously updated

MAKE IT TRAINABLE

The trace and eval loop that turns work into training data.

Governed access into internal systems through tools, CLIs and APIs; every run traced; every human correction captured for review. That is the raw material for a model of their own.

traces + corrections + private evals → their dataset, on their infra

LIVE TODAY

Profiling and the capture loop ship today and are accumulating in every deployment. Increasingly we solve these problems with our own specialised infra and agents, which is another way of saying **we're building AI that's good at building enterprise AI.**

THE HARD PROBLEMS AHEAD · WHAT THE PRE-SEED PAYS FOR

The version that survives the ugliest enterprise data, then the deep end: turning live capture into a **real training environment** — private evals scored against their outcomes, preference-grade correction data, rollout against the real systems — live reads, writes captured and scored — and RL where a workflow's volume earns it. Ours to own, because only we sit on the live capture.

Nothing new gets installed. Every component starts doing a second job.

The pieces that make AI work inside a company are the same pieces you need to train a model on how that company operates. Only the name of the job changes.

	Company Brain Component 01	Governed tool access Component 02	Sandboxes & deploy Component 03	Traces & corrections Component 04	Private evals Component 05
Today what it does for the transformation	Context an agent is grounded in, so answers cite the company instead of guessing.	Safe, audited actions inside real systems. No raw credentials, revocable.	Somewhere a non-engineer can ship an app or an agent without a ticket.	Observability. What ran, what it touched, what a person fixed afterwards.	Did this workflow actually work, measured against the outcome.
Tomorrow what the same thing becomes for training	The grounding corpus a model is fine-tuned against.	The action space a model is trained and tested in.	The rollout environment where attempts run safely, over and over.	Labelled data and preference signal, produced by their people doing real work.	benchmark, and the gate a candidate model has to pass. The only they own

WHY THIS IS INFRASTRUCTURE WORK

A world where every enterprise has AI of its own is not a world of a few frontier models doing everything. It is thousands of company-specific models, each needing somewhere to be built, grounded, run and measured. **That environment has to be repeatable, or it does not happen at all.**

WHAT IS LIVE, AND WHAT THE ROUND BUILDS

The capture layer ships today and accumulates in every deployment, because capture is the scarce part and only happens inside real work. Private evals are in build. Fine-tuning and distillation stack on top, with no rebuild and nothing new for the customer to install.

The same environment, drawn as the training layer.

One centralised store, four things acting on it. Three of the five ship today because the transformation needs them; the round builds the last two on top.

THE MODEL LAYER · WHATEVER THE COMPANY RUNS, BOUGHT OR TRAINED

Frontier APIs
Claude · GPT · Gemini

Open-weight bases
self-hosted

Fine-tuned models
theirs, per function

Distilled small models
the routine 80%

Candidate models
under evaluation

Model registry
versioned, promotable

One governed interface · train against it, evaluate against it, serve through it

SYNOS — THE PER-COMPANY AI TRAINING AND EVALUATION ENVIRONMENT

Self-hosted · nothing leaves the tenancy · the same install as job one

ONE STORE · THE SCARCE PART · ACCUMULATING NOW

**Company Brain** LIVE

The same centralised layer that grounds agents today is the corpus models are trained against tomorrow. One store, per company, on their own infrastructure.

entities & relations

traces & trajectories

skills & SOPs

corrections & labels

eval sets

curated datasets



Capture & labelling LIVE

What fills the brain. Every run traced with full lineage; every correction, approval and rejection captured from SMEs doing real work rather than an annotation vendor.



Rollout environment LIVE

Sandboxed execution against real systems, repeatable and reversible, on the same permissions and kill-switch the transformation runs on.



Eval harness IN BUILD

Task suites built from real traces, scored against the company's own outcomes. The gate a candidate model passes before promotion.



Fine-tune & distillation THE ROUND BUILDS

Supervised and preference tuning on open weights, distilled to small models per function, trained and served inside the customer's infrastructure.

DATA GOVERNANCE, UNCHANGED FROM JOB ONE

datasets stay in tenancy

no vendor egress

lineage on every example

per-function datasets

right to delete

GROUNDED IN THE SAME SYSTEMS THE TRANSFORMATION ALREADY CONNECTED

Warehouse

CRM & ERP

Docs & wikis

Slack & email

Internal APIs

Same connectors, permissions and audit

One environment, three phases. The first pays for the rest.

Everyone else authors a copy of the business and trains in the copy. This one instruments the original. The environment that makes AI work today is the one they need to train their own models tomorrow. Entry is how the position gets earned.

ENTRY

TODAY · REVENUE

Unblock the knowledge work

The layer installs in their cloud or fully air-gapped, and the company's non-engineers start doing real work on it, in the AI tools they already use, against their own systems. This is the pain they're paying to fix now, and **it's a good business on its own.**

revenue: paid POC → platform license + deployment

NEXT

SAME ENVIRONMENT

They train their own AI

Every run traced, every correction a label, private evals against **their** outcomes rather than public benchmarks. That is a **per-company AI training environment**, and what you need to fine-tune open-weight models on how this business operates.

revenue: data layer + evals + per-model / training runs

THE LONG ARC

WHAT WE'RE BUILDING FOR

Autonomous enterprise infrastructure

Most work running through agents and the software they wrote, on **models the company owns**. We operate the layer it lives on and get paid for what it produces.

the operating layer of the agent-native enterprise

The order is forced. **Capture is the scarce part and it only happens inside real work**, which is why the entry motion is the only way to earn the position the rest depends on, and it happens to be where the budget is today. What that makes us: **an AI-infrastructure play** — transformation is the entry motion, forward-deployed is only the delivery mechanics. One layer, three tenses, not three businesses.

AI earns autonomy here the way a new team member does.

Nobody hands a new hire the keys on day one. The environment makes that graduation explicit — for agents on frontier models today, and for the company's own models tomorrow.

01 · Rehearse

Sandboxes cut from the real systems.

Practice inside the customer's environment with no blast radius: dry-runs, scanned deploys, rehearsal copies we can spin up because the environment already knows their systems. A practice room, not the curriculum. [live today](#)

02 · Work, supervised

Real work, human sign-off.

Approval gates on actions that matter, every run traced, every correction captured from the person who owns the process. **This is where the training data comes from** — supervised real work, not a replica. [live today](#)

03 · Autonomous, audited

Runs alone once the evals clear their bar.

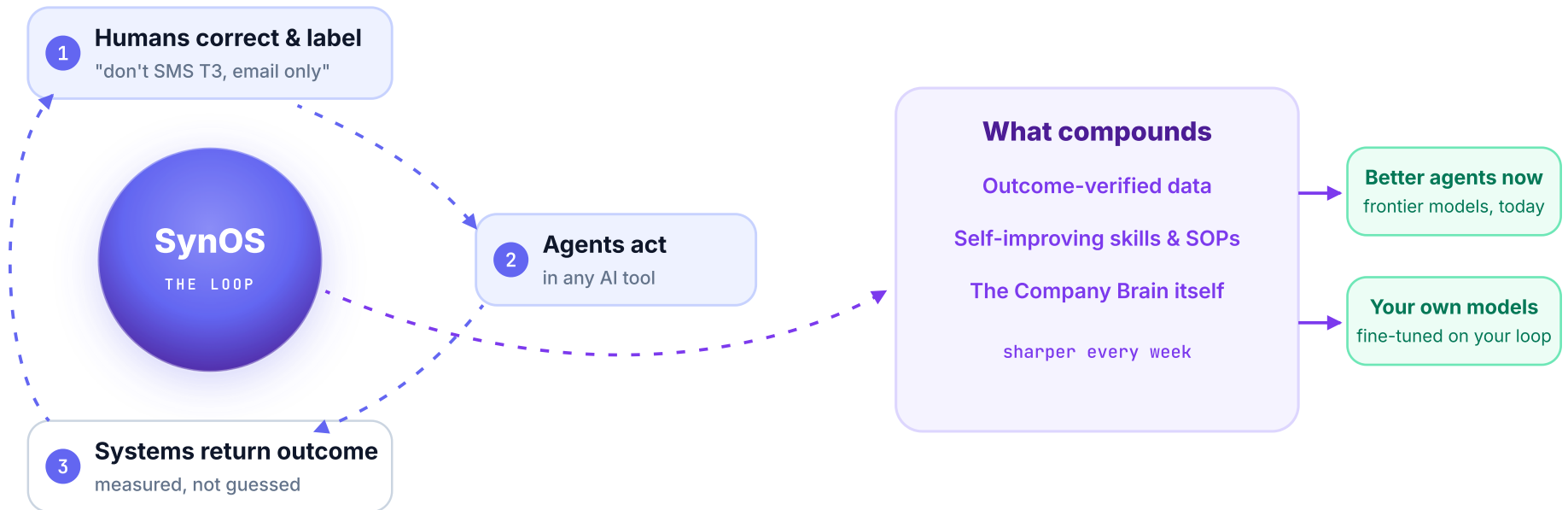
Private evals scored against their outcomes decide the promotion. Circuit breakers, kill switch, full audit stay on; authority is granted, revocable and re-checked.

[evals & mandates · the round builds](#)

The same ladder onboards both generations of their AI. An agent on a frontier model climbs it today; a model of their own climbs it tomorrow. And the onboarding record — every review, every correction, every outcome — **is the curriculum their models train on.** That is why the unblocking work and the training ground are one environment, not two products.

The moat is the loop: the record of how your company works, not the model.

Humans correct. Agents act. Systems return the measured outcome. Every turn builds data no public model can ever train on, and it pays off whichever model wins.



Live today: at a martech design partner, every marketer correction feeds one learning layer their whole platform gets smarter from. **This is data the model can never train on, and it accrues to the customer, on their infrastructure.**

Where we are: the capture layer, meaning tracing, corrections and agent-native storage, is live and accumulating in every deployment. Private evals are in build, and fine-tuning on the loop is the layer after that. Built data-first on purpose, because **capture is the scarce part** and training stacks on top with no rebuild. **Full flywheel in the appendix.**

Enterprises under real AI pressure, with no bench to build their way out.

Not the eng-heavy tech companies. The ones that **buy rather than build**, with a mandate they can't fulfil internally. Every deal we've closed fits this shape.

They own the domain expertise and the customer relationships, held by people who do not write code. What they lack is the engineering capacity to turn any of it into agents, and that gap closes only if the **non-engineers can build and run on the layer themselves**.

TRIGGER • CAPACITY

Thin AI bench

AI/ML req open for months, or re-posted; few or no ML titles; senior hires with no mid-level bench behind them.

TRIGGER • FAILED BUILD

Attempted, hasn't shipped

An AI initiative announced 6-12 months ago with nothing shipped, or a platform build that stalled after months. Enterprise AI licences are being cancelled at renewal because integration proved harder than it was sold as.

TRIGGER • POSTURE

Buy-first procurement DNA

An organisation that has always bought its systems. It will buy this one too, where an eng-heavy company would build and stall.

WHO SIGNS • THE OWNER OF THE MANDATE

Business or product leadership, with CEO air-cover

The person accountable for the AI outcome, not the VP Eng being asked to build it. Budget follows the mandate.

WHO WINS DAY ONE • THE SMES

The people who hold the knowledge

Ops, sales, marketing, finance and support author the agent workflows themselves, in plain English, without a ticket.

Two doors into the same layer. One qualifier opens both.

The same buyer feels it in two directions: outward at their product, inward at their operations. Same substrate, same buyer shape, same paid-POC contract.

DOOR 1 · OUTWARD — THEIR PRODUCT

Agentic transformation of their SaaS.

- Their competitors shipped AI features; their own platform build stalled. We make their product agent-native **in their own tenancy**, the enterprise-readiness layer they would otherwise spend two years building.
- Buyer: the AI/product owner carrying the roadmap, with CEO sponsorship. Expansion path: their clients, one at a time, on one substrate.
- Evidence today: **a martech platform. POC successful, moving to a paid client pilot.**

DOOR 2 · INWARD — THEIR OPERATIONS

A company brain their non-engineers can build on.

- Internal automation is blocked on engineering. We put the layer in and the **SMEs author the agent workflows themselves**. The knowledge holders build; engineering just sets the rails once.
- Enters at whatever hurts most today: a cost line, a sales-ops queue, a marketing pipeline. The pain is the **entry point**; the brain is the product.
- Evidence today: **a manufacturing enterprise and a US software company, both committed to paid contracts.**

Why this is one company, not two: both doors deploy the identical substrate: in-tenant deployment, governed writes back to the systems of record, and the per-customer record of corrections that compounds. Nothing forks. A door is a sales entry point; the asset being built is the same one, and it stays inside the customer's account.



Delivery compounds, and that's what keeps it a platform: an **FDE pair**, a domain expert plus a forward-deployed engineer, lands each account and runs the engagement on SynOS itself, so engagement N costs a fraction of engagement 1's human hours and every one mints reusable brains and skills. **Pipeline:** founder-led and advisor networks → design-partner referrals → embedded distribution → content and open-source inbound.

Three engagements live. All three verbally committed to paid contracts, in contracting now.

Names under NDA. Introduced live on request.

The proof story

A manufacturing enterprise, beginning with a Cloud FinOps Brain, plus agents that take critical DevOps actions. A committed paid POC.

Cloud-cost knowledge lived in a few engineers' heads. It moves onto the layer: billing data and people on one Company Brain, cited answers in plain English, every correction captured once. Agents then carry the DevOps actions on the same rails. **Committed paid monthly POC, kickoff underway**, and the expansion conversation is already about the next function.

committed paid monthly POC · Cloud FinOps Brain + DevOps agents · expansion in discussion

DOOR 2 · COMMITTED PAID POC · AIR-GAPPED ON-PREM

A US software company

Same landing template as the manufacturing enterprise, reused for a second close two weeks later, fully inside their own infrastructure. Post-POC pricing agreed.

DOOR 1 · AGENTIC SAAS TRANSFORMATION · POC SUCCESSFUL

A martech SaaS platform

Their product made agent-native on our rails, after their own platform build stalled. Four months on a warehouse-native assistant had not produced what they needed; ours ran in **two weeks**, on raw data of about 300 columns with no documentation. POC succeeded; moving to a paid pilot with a few of their own clients.

LIVE & DEMO-ABLE TODAY

Company Brain, living and continuously updated · access controls · agent-native storage · app deploy from Claude Code · works across AI tools via MCP · triggers · usage analytics & observability.

ROADMAP

The fine-tuning and deeper data-capture layers for custom / distilled models. The data is being captured today; training is the next layer.

Priced like infrastructure, because that's what it is.

The revenue ladder is the play restated in money. Platform fee on the footprint the environment covers — never per seat, because per-seat pricing punishes the thing we sell: more of the company on the layer.

LAND · TODAY

Paid POC → annual platform licence.

Live in weeks on one expensive workflow. Converts to a licence priced on the data footprint the environment covers, the way on-prem infrastructure has always been bought.

platform licence + deployment · grows with footprint, not headcount

EXPAND · THE MARGIN STORY

Licence grows. Delivery cost falls.

Each new function lands on rails already built, so the licence grows with coverage while templated delivery gets cheaper per engagement. That widening gap is what makes this a platform business, not a services one.

the number we hold ourselves to: delivery cost per engagement, falling

PHASE TWO · STACKS ON THE SAME INSTALL

The training layers become revenue lines.

Private eval suites, then fine-tuning and distillation runs on the captured data. Sold onto an environment already installed and already trusted, so there is no second procurement mountain.

evals · training runs · model ops — on the environment phase one paid for

WHY NOT PER SEAT

Agents don't hold seats, and the buyer's win condition is more people and more agents on the layer. The price follows the footprint of what the environment knows and governs, so our revenue grows exactly when the customer gets more value.

WHERE IT POINTS

As the layer proves what work produces, pricing moves toward outcomes. Pricing an outcome is underwriting, and you can only underwrite a business you understand — which is precisely what the environment accumulates.

Everyone owns one band. Nobody owns the three in the middle.

These are different categories doing different jobs, and each is good at its own. The gap between a company's systems and the AI its people already have is the part none of them was built for.

	Data platforms Databricks · Snowflake · Fabric	Enterprise search Glean	Workflow and iPaaS n8n · Workato	Agent platforms Dust · UnifyApps	SynOS today, and what it becomes
People and their AI tools Claude Code, Cursor, chat, apps		assistant		their agent	any harness
Execution and isolation tools, sandboxes, governed deploy			runs flows	runs agents	sandboxes · governed deploy
Governance identity, credentials, RBAC, audit, kill switch					agent acts as a revocable person
The data layer context today · training data tomorrow	catalog, once the data moves in	a copy of your documents		connectors	Company Brain profiled in place · every run traced, every correction captured
Systems of record ERP, CRM, warehouse, docs, tickets	move it here first		moves data		existing: stay yours, untouched · new apps & agents: born on the layer
Where it goes what this round builds	fine-tuning too — once your data lives in theirs				→ the same infra becomes the training ground for models they own

One data layer, two jobs. The Company Brain is the primary data layer: today it grounds every answer and agent in how the business actually runs, and the traces, corrections and evals it captures doing that are tomorrow's training data. Around it: an identity and permission model an agent can act through, and somewhere isolated to build and deploy. The data platforms own the bottom band and ask you to move everything into it first. Search reads a copy. Workflow tools execute without knowing what anything means. We deliberately do not touch your existing systems of record, and we do not need to own the tools your people already use. What your people build **new** gets an agent-native store inside the same environment — on your infrastructure, yours like everything else it accumulates — so the AI-era workflows never need a second procurement. Governance is the permission to act, isolation is where a rollout can safely run: built once for the transformation, and together with the Brain they are the place a company trains models of its own. The training compute underneath stays pluggable — Prime Intellect, Fireworks, or their own GPUs — the same neutrality we hold toward models. The environment is the part nobody else produces.

Three choices that are hard to copy, because each costs a competitor something real.

Not features. Each one requires giving up revenue, an architecture, or a category everyone else is chasing.

01 • The exit

You own what accumulates.

Every serious vendor lands with forward-deployed engineers. We are the one building for the handover: engineering keeps the rails, their domain experts operate and correct, and the record of how the company works stays on their side of the wall. A firm that bills for embedded engineers cannot copy this without cutting its own revenue. A multi-tenant platform cannot, because the learning lives in its product.

A martech platform said it back to us: "if tomorrow the customer says we don't want to continue, fine, we exit, but that data stays with us. You can't take my structured data which I have curated."

02 • Instrumented, not simulated

They author a copy and learn in the copy. We instrument the original.

Training environments are a funded category — Mercor and Fleet sell mocks to the labs, Prime Intellect sells the gym at a \$1B valuation. Every one of them, including the few that deploy the copy in your cloud, **authors a replica of your business and trains in the replica**. A replica cannot hold twelve years of exceptions in one company's order-to-cash. Ours is their own systems, instrumented during real work, with rewards from real corrections and outcomes — only possible because we were already doing the transformation. In phase two the gym vendors sit under us, not against us.

Why the order is forced: capture only happens inside real work. And the pull is cost — a lending platform's data lead: "it's exorbitant, the amount of tokens you consume; it just makes it unfeasible."

03 • Horizontal surface, vertical delivery

One expensive workflow at a time, on one layer.

Every engagement lands a single high-value piece of knowledge work with an ROI the buyer can name, the way a vertical product would. The layer underneath is the same one every time, so the second function costs less than the first and the tenth costs least of all.

This is what makes it infrastructure rather than a services business, and the automation curve is how we prove it.

WHAT WE DON'T CLAIM AS AN EDGE

A context layer, non-engineers building agents, bring-your-own-model, private-cloud and air-gapped deployment, forward-deployed delivery. Every serious vendor now has these, and a room that hears them pitched as differentiators learns something about the pitch rather than the product.

Every jump in model capability makes this layer more necessary, not less.

Enterprises don't stop at the model. They do **more** with it, and all of it still needs to know how the company operates and what it may touch. The test for built capital: take any one model away, and the company's capability survives, because it was never built into the model.

DEMAND

MORE CAPABILITY, MORE SURFACE

Better models mean more agents, and every one needs a company underneath it.

Each jump raises what an enterprise will hand over, and every one still needs the brain, the policies, the tools and something to evaluate against. **Capability doesn't grant permissions, and it doesn't know your exceptions.**

capability ↑ → more agents deployed → more of this layer per customer

DELIVERY

OUR OWN COST CURVE

The FDE work automates onto our own rails.

Today the motion is **platform plus a forward-deployed engineer**. Each jump moves more of what that engineer does into the platform, until delivery is agentic. Cost per outcome falls and the platform carries the growth.

platform + FDE → agentic FDE → platform-led delivery

VISION

THE ARRIVAL DATE MOVES

It pulls the autonomous enterprise closer.

Their own models, trained on their own loop, running more of the company. **Model progress shortens the distance to what we are actually building for**, on the environment we install today.

capability ↑ → the next phase arrives sooner, on the same environment

None of this asks you to bet that model progress slows down. **It asks you to bet that it continues**, which is the only part of this everyone already agrees on.

This layer sits at the intersection of three disciplines. We have all three.

Agentic analytics and semantic layers. On-prem enterprise infrastructure. Enterprise go-to-market in this region. Most teams attacking this problem have one of the three.

DATA & AGENTIC ANALYTICS

Sundial

Founder was CTO: built agentic analytics and semantic layers, and worked through the challenges of transforming a SaaS product to be AI-native.

ON-PREM ENTERPRISE INFRA

Nutanix

Eight years on distributed data systems. What on-prem demands of a product: air-gapped installs, upgrade paths, correctness with nobody from your team watching.

ENTERPRISE GO-TO-MARKET

Google

Amit, ex-Google, anchors go-to-market with strong India and SEA reach. Hiring from the same three pools is the first line in the use of funds.

Delivery is an FDE model, a domain expert plus a forward-deployed engineer. That embedded work is the wedge; **revenue scales with the platform**, not with hours billed, and what the FDE team learns becomes brains and skills on the platform.

We run SynOS on SynOS. Our own GTM, research and operations run as agents on the layer. It's why, four months in, there's a live platform and three committed engagements.

India first, US in test. The buyer profile is not geography-specific, and the US wedge is enterprise outside the Bay Area. Which market we scale into is an output of the GTM test.

A pre-seed, to turn three committed contracts into a repeatable motion and a training layer.

Three things, in this order. The first is the constraint on everything else.

01
FIRST LINE, FIRST RUPEE

Senior founding hires

Engineering deep enough to own the messy-data problem, go-to-market, and domain SMEs who can run an FDE pair. The binding constraint on everything else.

the team is the use of funds

02
SIX MONTHS, IN PARALLEL

India GTM at scale, US GTM opened

Scale the India motion that is already converting; open the US through enterprise outside the Bay Area. Three routes carry kill thresholds written before the data, then we concentrate.

one entry wedge, chosen on evidence

03
STACKS ON WHAT SHIPS

R&D: the fine-tuning infra and environment

Private eval suites hardened first, then fine-tuning and distillation on top of the capture already running. This is what turns the transformation environment into the one they build their own AI in.

phase one funds it, phase two compounds it

WHAT IS TRUE WHEN THIS ROUND IS SPENT

Revenue

Committed contracts paid and referenceable

Repeatability

Delivery cost per engagement instrumented, falling

Product

Eval + fine-tuning infra live on real capture

Phase-two proof

First training run on one customer's own data

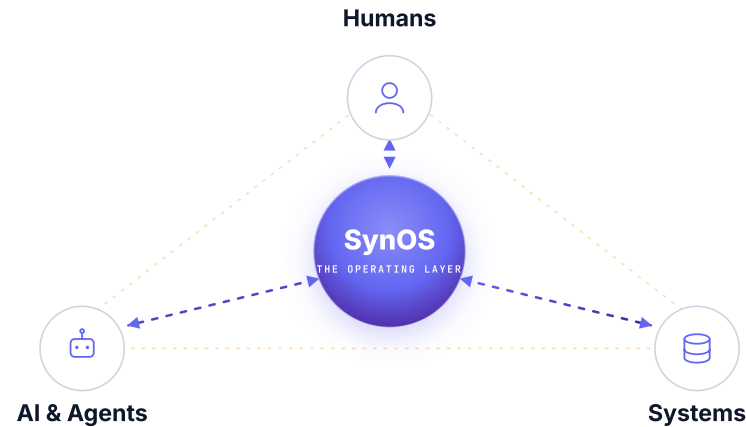
Geography

India motion scaled; first US logo outside the Bay Area

What it buys, plainly: **a team, a proven way to repeat the deals we already have, and the layer that makes the second phase real.** Terms and the current round structure on a call.

SYNOS

Make your enterprise agent-native, today. Build your own AI, and own it, tomorrow.



The layer where **humans, AI and systems work at light speed**, creating value at the edge of what current AI capabilities can achieve.

Raising our pre-seed. Let's talk.

Self-hosted · model-agnostic · works with any AI stack · built for the knowledge workers, not just the engineers · your data, your moat

END OF THE MAIN DECK

Appendix

Detail on request.

What runs on the layer, the status of the two doors, where value gets created at the edge of model capability, the market evidence, the SaaS shift, plain-English build and deploy, and the full data flywheel.

Security and readiness overview, architecture deep-dive and a live demo available on a call.

One environment, under everything your teams already use.

Your teams go on building agents wherever they do today. SynOS is the environment **underneath** them, so the company gets one governed place where its AI runs, learns and is measured.

WHERE AGENTS GET BUILT · WHATEVER YOUR TEAMS ALREADY USE

Claude Code

Codex

Cursor

n8n

Copilot

your own apps

SYNOS — ONE SELF-HOSTED LAYER · INSTALLED ONCE · MODEL-AGNOSTIC

Company Brain self-curating, not static RAG	Deploy & share sandboxed, governed, no ticket	Access & permissions agents acting on behalf of people	Agent-native storage where agent output actually lives	Evals & observability every run traced and improvable
---	---	--	--	---

YOUR SYSTEMS OF RECORD · STRUCTURED AND UNSTRUCTURED

Warehouses

SaaS apps

Internal data stores

ERP

Communication layers

Docs & wikis

Every enterprise going agent-native ends up building these same five things internally, separately and slowly. **A martech platform spent four months on a platform build that never shipped. Three India tech majors went build-first and still aren't in production.** That work is the product.

Both doors have converted. The next quarter picks the one we scale.

Demand is real on either side, and the criteria are written down before the data: **one entry wedge**, then repeated deliberately.

1

account through **Door 1**. The POC succeeded and it is moving to a paid client pilot.

OUTWARD · PRODUCT

2

accounts through **Door 2**. Both committed to paid contracts, and the landing template was reused for the second close two weeks after the first.

INWARD · OPERATIONS

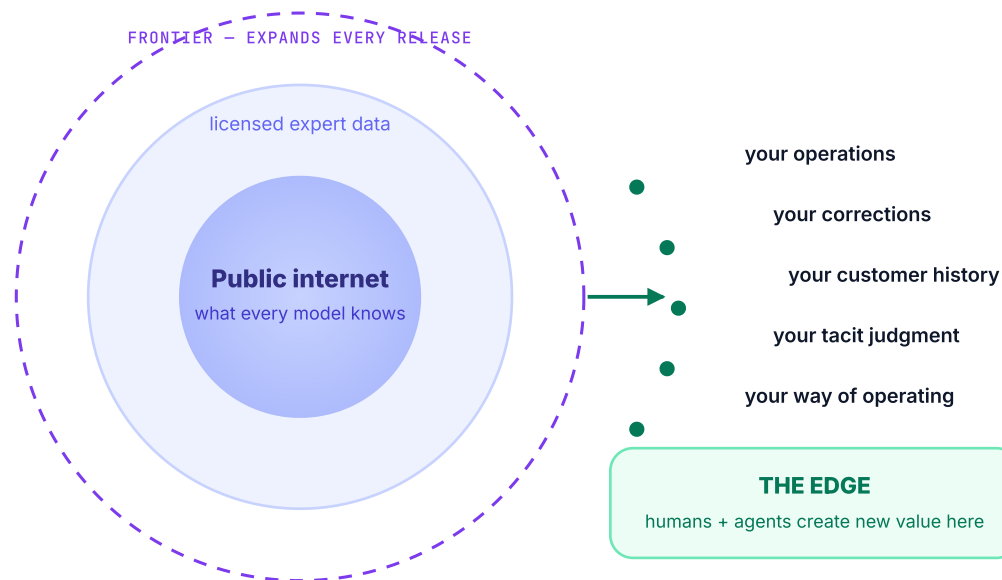
We decide on evidence rather than instinct, and the criteria are written down before the data.

The gate we are running to: **which door produces a second customer from the same template with materially less delivery effort than the first**, and **which one expands inside the account without new engineering**. That is the door that gets the spend. A pre-registered demand test with kill thresholds runs alongside the live POCs, with control arms that isolate what sells cold.

The live POCs **are** the experiment, so the answer comes out of the deployments rather than ahead of them. The same playbook is being run in the US and in India in parallel, so the market is an output of the test too. **We expect to know by the end of Q3.**

New value is created at the edge of what models know.

Humans and agents, working together on one layer, create what no model holds, and everything they create teaches your AI. Humans move at AI speed; your AI learns your company.



HUMANS GET FASTER

Knowledge work runs at AI speed, on the company's own context rather than generic answers.

THE EDGE IS YOURS ALONE

What's created at the boundary, meaning decisions, corrections and outcomes, exists in no model and no competitor.

CREATING VALUE MAKES YOUR AI BETTER

Every piece of edge-work feeds the loop: your agents, skills and brain sharpen with the work itself.

"You can offload a task, or even a job — **you can never offload your learning.**" As models commoditize expertise, the durable advantage moves from the model to the learning loop you own. — Satya Nadella, 2026

Enterprises are all trying this. Most fail the same way.

The failure mode is universal, and it names exactly the layer we sell.

40%

of agentic AI projects will be cancelled by 2027. "Agent sprawl," and the integration cost is never in the pilot budget.

GARTNER · 2025

95%

of enterprise GenAI pilots deliver no P&L impact. The gap is the learning and integration layer rather than model quality.

MIT · 2025

68%

of employees already use AI tools without IT approval; only ~10% of F500 have any agent-governance strategy.

CSA · 2026

And the model future is hybrid, including open-source.

~80% of enterprise use cases already run well on open-source and smaller fine-tuned models. Enterprises will mix frontier, open-weight and their own fine-tuned models, for cost, sovereignty and control. Every mix needs the same two things: **a model-agnostic layer, and the company's own data as fuel.** That's us, twice.

Vendor-built platforms succeed **2x as often** as internal builds (MIT). The layer is the missing product.

Every company operates differently. SaaS forced them to operate the same.

The old deal: buy the common 80%, bend your operations to fit it, and never get the 20% that's actually you. Agents end that deal.

THE OLD WORLD · RENTED SAAS

You adjusted to the software.

- ✗ One-size-fits-none workflows: your ops bent to the vendor's shape.
- ✗ Seats for the common 80%; the custom 20% was never the vendor's job.
- ✗ Your data and your process knowledge, held in someone else's cloud.

THE NEW WORLD · ON YOUR LAYER

The software understands you.

- ✓ **Custom apps and always-on agents** that know how **you** operate, described in plain English, live in days.
- ✓ Built on your company's own memory, rules and corrections: the 20% you always needed, finally yours.
- ✓ Owned by you, on your infrastructure, and it gets smarter with every use.

*SynOS is the layer that makes **custom finally cheaper than SaaS**. Companies build software that understands them instead of adjusting to software that doesn't.*

Once the hard layer exists, the "magic" is just a feature.

Use whichever agent or app builder your teams already love. **Build, deploy and share in an afternoon onto your company's shared surface**, where every skill and correction compounds instead of dying on a laptop. All of that is easy once the memory, storage, permissions and sandboxes exist underneath. It is why we built the layer first.

PLAIN ENGLISH → AGENT

Describe the job. Get an agent.

A knowledge worker describes the task in plain English → it becomes a versioned skill → one click promotes it to an always-on agent with triggers, tools and guardrails.

"every Monday, check spend spikes and post to Slack"
→ running agent

PLAIN ENGLISH → APP

Describe the app. Get a live URL.

Describe the dashboard or tool → it's built and deployed to a live, sandboxed URL with governed data access. No ticket, no engineering queue.

idea → working app · sandboxed · same afternoon

TRUST, GRADUATED

Run it → schedule it → set it free.

Every automation climbs a trust ladder: run it yourself → scheduled with human review → autonomous with audit and a kill-switch. Autonomy never means unsupervised.

assisted → reviewed → autonomous · governed at every step

Point solutions sell each of these as a product. On SynOS they're **features of the layer**, grounded in the company's own memory and loop.

One layer. Every function is a template away.

We enter through one wedge, but the surface underneath is the same for all of it. Every use case feeds the same Company Brain and the same loop.

FLAGSHIP

Company Brain

One living memory of how the company operates. Tribal knowledge captured once, used by every team and every agent.

PRODUCT

Agent-native product transform

Turn an existing product into an agent-native platform on our rails. Live with a martech platform today.

FUNCTIONS

Function brains

Sales Ops · Marketing · FinOps / Cloud DevOps. Pre-built starting points, live in weeks, then tuned to how you operate.

OPERATE

From the tools they already use

Claude Code, Codex, Cursor. Non-engineers author skills in plain English and operate the whole layer.

BUILD

Apps & always-on agents

Deploy governed apps and always-on agents on the same rails: sandboxed, audited, with real data access.

AND BEYOND

Your domain

The layer is horizontal. Each new function or domain is a template away, on the same compounding memory.

Every run is traced. Every trace is future training data.

The tracing layer shipping today is quietly building each customer's fine-tuning dataset, the raw material for their own models tomorrow. We built it data-first on purpose.

01 · Trace

Every run captured

Inputs, tool calls, decisions, outcome: full lineage on every agent run.

● LIVE TODAY

02 · Label

Corrections captured

Every human correction and approval is captured and reviewed in the flow of work — the material labels are built from.

● LIVE TODAY

03 · Eval

Private evals

Agents scored against **your** outcomes, not public benchmarks. Ground truth only you own.

● IN BUILD

04 · Dataset

Outcome-verified data

Curated per company: traces that worked, corrections that fixed, evals that prove it.

● ACCUMULATING NOW

05 · Fine-tune & distill

Your own models

Custom and distilled small models for your workflows, trained on the loop and run on your infra.

◆ ROADMAP

COST

Small models carry the routine

Distilled models run the ~80% of routine work at a fraction of frontier-token cost. The open-source future, powered by your data.

SOVEREIGNTY

Your data, your models, your infra

Nothing trains a public model. Your models live in your cloud, so you can swap providers freely without losing what you've learned.

MOAT

A dataset nobody can buy

Minted from your own operations and corrections. The one asset a competitor or a lab cannot replicate.

Where we are: the data layer (tracing, corrections, agent-native storage) is live and accumulating in every deployment, and the eval and training layers stack on top of it next. Built data-first on purpose, because **capture is the scarce part** and training needs no rebuild.

The categories we get compared to each unblock one piece.

We unblock humans, agents and the enterprise **together**. The honest map against the nearest tools, and who each was built for.

YOU'LL BUCKET US WITH...

Eval & observability platforms

agent tracing & eval tooling

Enterprise search & RAG

knowledge retrieval products

Context / memory layers

agent memory stores

Tool proxies & MCP gateways

connectivity & governance pipes

AI workflow builders

low-code automation platforms

Agent harnesses

Claude Code · Codex · Cursor

WHAT THEY UNBLOCK

Scoring recorded outputs against datasets an engineer authors.

Finding documents across silos.

Memory for one agent, one app.

Governed pipes to real systems.

Automating one defined flow.

A very capable individual agent.

WHAT STAYS BLOCKED

No environment attached: they score what an agent said; we execute against the real systems and score what it did, authored by the SME, judged against the company's own outcomes.

No hands: can't act in systems, no outcome to learn from, so the loop never starts.

A component, not a layer: no entity resolution across systems, no governance, no deploy, and framework lock-in.

Pipes without a brain: no shared context, no skills, nothing compounds between calls.

Every run starts from zero: no company brain, rigid graphs, per-vendor lock-in.

No company underneath: no shared brain, no safe data access, nowhere governed to deploy, no way to share wins.

SYNOS · BUILT GROUND-UP FOR NON-ENGINEERING KNOWLEDGE WORK, PROVIDER-AGNOSTIC

The full agent & agent-data infra stack: Company Brain, permissioned access, sandboxes, governed deploy, and the learning loop, all under any AI tool, on your infra. Engineering sets the rails once; the domain experts operate on them. An agent- and data-infra play: every run and correction accumulates and compounds into your enterprise's own data.

BUILT FOR A DIFFERENT PERSON

Every category above is built for engineers. The developer is the user, and the enterprise's domain experts are downstream of a ticket. That is the gap: the knowledge lives with people who cannot code, and the value only unlocks when **they** can build and run on the layer safely. Not all rivals, either: harnesses, search and memory tools plug into us over MCP.

WHY THE LABS WON'T BUILD IT

It must be self-hosted, neutral across every AI tool, and made of **your** corrections: three things a model vendor structurally will not do.

Everyone lands the same way. Almost nobody leaves.

Context layers, non-engineer authoring and forward-deployed delivery are table stakes now. What separates vendors is what the customer owns and operates a year later.

↑ the learning stays in your company · in the vendor's product ↓

COHERE NORTH · CONDUCT · ERAGON · SARVAM

Sovereign, but still delivered to you

Runs on your infrastructure, sometimes air-gapped. Their people build it and their people keep running it.

SYNOS – THEN: THEY TRAIN THEIR OWN MODELS ON IT

Yours to operate

We install the layer, land the first workflow, then hand the controls to their domain experts and step back. What accumulates is theirs.

DISTYL · SIERRA · DECAGON · EMA · MOVEWORKS · THE SIS

Outcome delivered, vendor stays

Excellent products. Engineers embedded for the contract, or an outcome bought per vertical. Nothing transitions to the customer.

UNIFYAPPS · DUST · GLEAN · WORKATO · N8N

You operate it, the learning doesn't stay

Real platforms your team runs. Multi-tenant by design, so what your company teaches it lives in their product, not yours.

← A solution delivered to you

Infrastructure your own people run →

The top right is empty, and it is hard to move into. Reaching it needs a delivery model that lets the vendor withdraw and an architecture where the corrections live on the customer's side. The firms on the left bill for the people who stay. The platforms below are multi-tenant, so the learning cannot sit with one customer.

Enterprises handed agents to everyone. The floor gave way.

They got the tools without company knowledge or safe rails. **Every wall below is an engineering problem standing between a domain expert and their own work, and everything built for this market assumes that person is an engineer.**

"I built an app. Now deploy and share it."

No safe, sandboxed host. Engineering blocks it.

"I wrote a skill. Can I share it across teams?"

No reuse. No self-improvement. No visibility.

"Where does my agent's output live?"

Random Sheets & DMs. No governed storage.

"My skill needs the warehouse & SaaS data."

Raw creds in chat. Engineering blocks it.

"What's the source of truth across systems?"

No clean, shared answer. Everyone re-derives.

"Can it get smarter over time?"

Corrections die in chat history. Nothing compounds.

Six angles, one problem: **agents can work now, but the enterprise has no layer for humans and agents to work on together.**

Each vendor's AI exists to lock in its own platform.

Enterprises already run multiple clouds, multiple SaaS platforms, multiple model providers, and no CIO will standardize the company's AI on a stack that belongs to one of them.

The enterprise stack today: every layer pushing its own AI

HYPERSCALERS

each cloud → its own AI suite

tied to its own compute

SAAS PLATFORMS

every CRM / ERP → its own copilot

locked to its own data

MODEL PROVIDERS

each lab → its own enterprise suite

on its own models only

Each one's AI exists to defend its own platform, which makes it structurally unable to be the layer that spans all of them.

The neutral layer is the only one everyone can meet on.

Model-agnostic, tool-agnostic, self-hosted on the customer's infra. **The test: take any one model away, and see whether the company's capability survives.** On SynOS, the brain, skills, evals and agents survive the swap. Built capital, not rented intelligence.

And it is why they stay. Integration lock-in is dying: a competitor can have agents rewrite connectors in an afternoon, and we won't claim otherwise. What can't be rewritten is **what a company's own people taught the environment.** Two years of corrections and exception handling take another two years to earn.

any cloud

any model

any agent stack

your infra

your data

WHY US

This layer is the intersection of three disciplines: agentic analytics and semantic layers, on-prem enterprise infrastructure, and enterprise go-to-market in this region. **And the pattern rhymes:** the last time enterprises faced a transformation they could not execute alone, the platform that carried them across was on-prem-first, solved genuinely hard infrastructure problems, and stayed deliberately neutral to the layer above: hypervisors then, models now. A structural parallel rather than an identity claim.